

OLYMPUS CLIMB 2026

Documento de Arquitetura-Alvo

1. Visão do Produto

1.1 O que é o Olympus Climb

O Olympus Climb será uma **plataforma pessoal de operação de carreira assistida por IA**, desenhada para dois objetivos simultâneos:

1. **servir como laboratório real de estudo de IA avançada**, especialmente agentes, skills, tool calling, MCP, memória, avaliação e automação assistida;
2. **operar o funil real de prospecção e candidatura a vagas**, aumentando qualidade de decisão, velocidade de execução e consistência estratégica.

1.2 O que o projeto não será

O Olympus Climb não será tratado como:

- um scraper isolado;
- um app mobile isolado;
- um dashboard isolado;
- uma coleção de automações avulsas.

Essas peças existirão, mas como partes subordinadas a um sistema maior.

1.3 O que o projeto será

O Olympus Climb será tratado como:

um sistema agentic para descobrir oportunidades, enriquecer contexto, avaliar compatibilidade, decidir estratégia, executar candidaturas com controle humano e aprender continuamente com os resultados.

2. Princípio Arquitetural Central

2.1 Mudança de eixo

A arquitetura antiga orbitava em torno de:

- scraper;
- backend;
- mobile;
- dashboard.

A arquitetura nova passa a orbitar em torno de:

- **core de dados confiável;**
- **runtime agentic;**
- **skills e tools plugáveis;**
- **memória operacional;**
- **camada de aprovação humana;**
- **interfaces de operação.**

2.2 Princípio de desenho

Toda decisão de arquitetura deve obedecer à seguinte regra:

o valor do sistema não está em coletar vagas; está em operar com inteligência o ciclo completo de descoberta, análise, decisão, adaptação e execução de candidatura.

2.3 Consequência prática

Qualquer módulo novo deve responder a pelo menos uma destas funções:

- aumentar a qualidade da decisão;
 - aumentar a qualidade da execução;
 - reduzir retrabalho;
 - preservar contexto;
 - tornar o agente mais confiável;
 - ampliar a capacidade do sistema de aprender com o próprio histórico.
-

3. Stack-Alvo do Sistema

3.1 Stack principal

3.1.1 Frontend operacional

- **React Native + Expo**
- **TypeScript**
- **NativeWind** para mentalidade utility-first e reuso de tokens
- camada de componentes reutilizáveis inspirada no que já funcionou nos projetos anteriores

3.1.2 Backend principal

- **Node.js**
- **NestJS**
- **Fastify**
- **Prisma ORM**
- **PostgreSQL**

3.1.3 Infra de processamento

- **Redis**

- BullMQ
- PM2

3.1.4 Observabilidade e qualidade

- Jest
- Artillery
- TypeDoc
- logging estruturado
- tracing de execuções agentic

3.1.5 Camada agentic

- arquitetura baseada em **Responses API**
 - uso de ferramentas integradas e tool calling
 - suporte a **remote MCP**
 - desenho compatível com **Agents SDK** e orquestração moderna de agentes ([OpenAI](#))
-

4. Objetivos de Engenharia

4.1 Objetivo funcional

Permitir que o sistema:

- descubra vagas;
- entenda as vagas;
- relacione essas vagas ao teu perfil;
- proponha estratégia;
- execute ações;
- registre tudo;
- aprenda com o que aconteceu.

4.2 Objetivo técnico

Construir um sistema:

- modular;
- auditável;
- plugável;
- estável;
- escalável;
- observável;
- seguro para evolução de longo prazo.

4.3 Objetivo de estudo

Usar o projeto como veículo para praticar, em contexto real:

- agentes especializados;

- tool calling;
 - MCP;
 - memória;
 - evaluation loops;
 - approvals;
 - automação com browser;
 - workflows agentic com rastreabilidade.
-

5. Capacidades-Mãe do Produto

O Olympus Climb será organizado por **capacidades**, não por cronograma.

5.1 Descoberta

Capacidade de encontrar vagas relevantes em múltiplas fontes.

5.2 Normalização

Capacidade de transformar dados heterogêneos em uma estrutura única e confiável.

5.3 Enriquecimento

Capacidade de buscar contexto adicional sobre vaga, empresa, stack, risco e sinais relevantes.

5.4 Match

Capacidade de comparar oportunidade com perfil, histórico, objetivos e restrições.

5.5 Estratégia

Capacidade de decidir o que fazer com a vaga:

- aplicar;
- ignorar;
- guardar;
- revisar;
- adaptar currículo;
- adaptar mensagem;
- pedir aprovação humana.

5.6 Execução

Capacidade de realizar ações concretas:

- preencher formulário;

- responder perguntas;
- gerar e-mail;
- gerar mensagem;
- registrar prova de envio;
- agendar follow-up.

5.7 Memória

Capacidade de preservar histórico útil e reusar conhecimento acumulado.

5.8 Observabilidade

Capacidade de explicar:

- o que foi feito;
 - por que foi feito;
 - por qual agente;
 - com quais inputs;
 - com quais tools;
 - com qual resultado.
-

6. Arquitetura Lógica do Sistema

6.1 Camadas macro

6.1.1 Core Data Layer

Fonte de verdade do sistema.

Responsável por:

- entidades;
- relacionamentos;
- histórico;
- documentos;
- trilha de auditoria;
- preferências;
- resultados de execuções.

6.1.2 Agent Runtime Layer

Camada onde agentes são executados, delegados, rastreados e controlados.

Responsável por:

- orquestração;
- handoffs entre agentes;
- tool calling;

- retries;
- guardrails;
- approvals;
- sessões;
- contexto de execução.

6.1.3 Skills / Tools Layer

Camada das capacidades invocáveis.

Responsável por:

- integração com web;
- parsing;
- scoring;
- análise documental;
- automação de navegador;
- leitura de e-mails;
- atualização de agenda;
- persistência contextual.

6.1.4 Human Oversight Layer

Camada de controle humano.

Responsável por:

- aprovar ações sensíveis;
- revisar saídas;
- corrigir decisões;
- bloquear execução indevida.

6.1.5 Operator Interfaces

Camada das interfaces de uso.

Responsável por:

- app mobile;
- eventual painel web;
- inbox de tarefas;
- fila de aprovação;
- visão de pipeline;
- consulta de histórico.

7. Core de Dados

7.1 Princípio

O banco não é depósito passivo.
Ele é o **estado canônico** do sistema.

7.2 Entidades centrais propostas

7.2.1 JobSource

Representa a origem da vaga.
Exemplos:

- LinkedIn
- Gupy
- Greenhouse
- Lever
- página própria
- indicação manual
- importação por documento

Campos sugeridos:

- id
- nome
- tipo
- baseUrl
- status
- confiabilidade
- regras de coleta

7.2.2 Company

Representa a empresa.

Campos sugeridos:

- id
- nome
- site
- linkedinUrl
- segmento
- porte
- país
- reputação agregada
- notas internas
- flags de risco
- data de última análise

7.2.3 JobPosting

Representa a vaga normalizada.

Campos sugeridos:

- id
- sourceId

- companyld
- externalld
- título
- descrição bruta
- descrição normalizada
- senioridade inferida
- modalidade
- localização
- faixa salarial
- moeda
- stack extraída
- requisitos extraídos
- diferenciais extraídos
- status interno
- score global
- score de match
- score de risco
- publishedAt
- discoveredAt
- updatedAt
- url
- hash de deduplicação

7.2.4 ResumeProfile

Representa uma variante de currículo.

Campos sugeridos:

- id
- nome da variante
- foco
- stack alvo
- resumo profissional
- versão atual
- documento associado
- tags
- ativo ou inativo

7.2.5 CoverAsset

Representa material de adaptação.

Campos sugeridos:

- id
- tipo
- template-base
- variante
- texto gerado
- contexto utilizado
- vaga relacionada
- data de geração

7.2.6 Application

Representa uma candidatura.

Campos sugeridos:

- id
- jobPostingId
- resumeProfileId
- status
- canal
- data de envio
- evidência
- respostas enviadas
- abordagem usada
- follow-up previsto
- outcome final

7.2.7 AgentRun

Representa uma execução agentic rastreável.

Campos sugeridos:

- id
- tipo de agente
- objetivo
- contexto de entrada
- tools usadas
- custo
- duração
- resultado
- status
- logs
- aprovações exigidas
- aprovação recebida

7.2.8 DecisionLog

Representa a justificativa das decisões.

Campos sugeridos:

- id
- jobPostingId
- agente responsável
- decisão
- rationale
- critérios considerados
- confiança
- revisão humana
- timestamp

7.2.9 MemoryEntry

Representa memória útil do sistema.

Campos sugeridos:

- id
- tipo
- conteúdo
- escopo
- relevância
- origem
- data de criação
- validade
- embeddings associados

7.2.10 SkillExecution

Representa execução de skill específica.

Campos sugeridos:

- id
 - skill
 - inputs
 - outputs
 - duração
 - erro
 - agente chamador
 - recurso afetado
 - timestamp
-

8. Runtime Agentic

8.1 Princípio

O runtime agentic não deve ser um “agente Deus”.
Ele deve ser composto por **agentes especialistas**, com responsabilidades estreitas.

8.2 Agentes base propostos

8.2.1 Scout Agent

Missão:
encontrar oportunidades.

Responsabilidades:

- buscar vagas em fontes conhecidas;
- buscar vagas novas na web;
- identificar duplicatas;

- priorizar coleta.

Não decide aplicação.

8.2.2 Normalizer Agent

Missão:
padronizar dados.

Responsabilidades:

- converter dados crus em schema interno;
- inferir campos ausentes;
- separar ruído de conteúdo útil;
- gerar hash de deduplicação.

8.2.3 Enrichment Agent

Missão:
enriquecer o contexto da oportunidade.

Responsabilidades:

- levantar contexto da empresa;
- levantar indícios de ambiente;
- levantar stack provável;
- levantar sinais de maturidade;
- levantar risco percebido.

8.2.4 Match Agent

Missão:
comparar oportunidade e perfil.

Responsabilidades:

- calcular aderência técnica;
- calcular aderência estratégica;
- considerar localização, modalidade, momento de carreira, objetivos e restrições;
- produzir score explicado.

8.2.5 Strategy Agent

Missão:
definir a melhor ação.

Responsabilidades:

- propor aplicar;
- propor descartar;
- propor guardar;
- propor adaptar currículo;
- propor escrever abordagem específica;
- definir necessidade de aprovação humana.

8.2.6 Execution Agent

Missão:
executar tarefas concretas.

Responsabilidades:

- preencher formulários;
- gerar respostas;
- enviar e-mails;
- registrar evidências;
- atualizar status no sistema;
- disparar follow-up.

8.2.7 Memory Agent

Missão:
preservar e recuperar contexto útil.

Responsabilidades:

- consolidar aprendizados;
- recuperar histórico relevante;
- associar contexto a empresas, vagas e decisões;
- reduzir repetição burra.

8.2.8 Review Agent

Missão:
revisar saídas de alto impacto.

Responsabilidades:

- validar coerência;
 - detectar alucinação operacional;
 - detectar contradição com histórico;
 - bloquear execução suspeita;
 - pedir revisão humana quando necessário.
-

9. Skills

9.1 Definição

Skills serão tratadas como **unidades formais de capacidade operacional**.

Cada skill deve ter:

- propósito único;
- contrato de entrada;
- contrato de saída;

- regras;
- limites;
- logs;
- testes;
- exemplos reais;
- fallback.

9.2 Estrutura mínima por skill

Cada skill deverá conter:

- `README.md`
- `input.schema.ts`
- `output.schema.ts`
- `handler.ts`
- `evals/`
- `fixtures/`
- `tests/`
- `examples/`

9.3 Skills candidatas iniciais

9.3.1 `search_jobs`

Busca oportunidades em fontes externas.

9.3.2 `open_job_post`

Abre e captura conteúdo bruto da vaga.

9.3.3 `extract_job_requirements`

Extrai requisitos, diferenciais, senioridade, modalidade e stack.

9.3.4 `normalize_job_post`

Converte para o schema canônico.

9.3.5 `deduplicate_job_post`

Compara com vagas já existentes.

9.3.6 `analyze_company_context`

Busca contexto institucional e sinais de risco.

9.3.7 `score_job_fit`

Calcula aderência da vaga ao teu perfil.

9.3.8 **select_resume_variant**

Escolhe a melhor variante de currículo.

9.3.9 **generate_application_assets**

Gera respostas, sumários, mensagens e assets de candidatura.

9.3.10 **fill_application_form**

Executa preenchimento assistido.

9.3.11 **send_application_email**

Envia e-mail quando aplicável.

9.3.12 **save_application_evidence**

Registra prints, links, ids, mensagens e comprovantes.

9.3.13 **schedule_followup**

Agenda acompanhamento.

9.3.14 **record_decision**

Salva rationale e decisão tomada.

9.3.15 **update_memory**

Consolida aprendizado da execução.

10. MCP no Projeto

10.1 Papel do MCP

O MCP será tratado como **camada de integração padronizada** para conectar o runtime agentic a sistemas, ferramentas e dados externos. O protocolo define um modelo de integração com recursos, ferramentas e prompts, e os MCP servers expõem capacidades padronizadas para aplicações de IA. ([Model Context Protocol](#))

10.2 Por que usar MCP aqui

Porque ele favorece:

- desacoplamento;

- padronização de integrações;
- reuso;
- plugabilidade;
- evolução sem contaminar o core do sistema.

10.3 MCPs candidatos

- Gmail / e-mail
- Calendar
- Google Drive / documentos
- storage de arquivos
- browser automation onde aplicável
- planilhas / tracking
- repositórios / notas / knowledge base

10.4 Regra de adoção

MCP só entra quando:

- reduz acoplamento;
 - padroniza algo que seria gambiarra customizada;
 - permite reuso futuro;
 - não complica desnecessariamente a operação.
-

11. Interfaces de Operação

11.1 App mobile

O app React Native + Expo será o **cockpit operacional** do sistema.

Funções principais:

- inbox de oportunidades;
- fila de revisão;
- fila de aprovação;
- visão do pipeline;
- leitura do rationale dos agentes;
- comparação vaga x perfil;
- consulta de empresa;
- consulta de histórico;
- quick actions.

11.2 Painel web futuro

O painel web não é obrigatório no primeiro momento, mas a arquitetura deve deixá-lo fácil de acoplar.

Uso provável:

- administração;
 - observabilidade;
 - debug;
 - relatórios;
 - análise profunda de agent runs;
 - gestão de skills e evals.
-

12. Aprovação Humana

12.1 Princípio

O sistema pode automatizar muito, mas não deve automatizar cegamente.

12.2 Ações que exigem aprovação

Por padrão, exigem aprovação humana:

- envio de candidatura;
- envio de e-mail;
- envio de mensagem para recrutador;
- alteração destrutiva em dados;
- uso de uma nova abordagem não validada;
- resposta a pergunta aberta de alto impacto.

12.3 Ações que podem ser autônomas

Podem ser autônomas:

- descoberta;
 - parsing;
 - normalização;
 - enriquecimento;
 - scoring;
 - deduplicação;
 - ranking;
 - agendamento interno;
 - atualização de memória e logs.
-

13. Memória do Sistema

13.1 Definição

A memória deve ser útil, seletiva e auditável.

13.2 O que guardar

- empresas já analisadas;
- vagas semelhantes;
- respostas que funcionaram;
- rejeições relevantes;
- padrões de alinhamento;
- preferências reais;
- sinais de risco recorrentes;
- versões de currículo e seus resultados.

13.3 O que não guardar sem critério

- lixo de execução;
- contexto redundante;
- outputs transitórios sem valor futuro;
- inferências frágeis tratadas como verdade.

13.4 Princípio de memória

Memória existe para:

- melhorar decisão;
 - reduzir repetição;
 - evitar retrabalho;
 - aumentar personalização;
 - sustentar continuidade.
-

14. Observabilidade e Auditoria

14.1 Tudo importante deve ser explicável

Toda ação relevante deve responder:

- quem executou;
- quando executou;
- com qual contexto;
- com quais tools;
- com qual decisão;
- com qual saída;
- com qual custo;
- com qual evidência.

14.2 Logs mínimos

- logs de agente
- logs de skill
- logs de fila
- logs de aprovação
- logs de execução externa
- logs de erro
- logs de decisão

14.3 Métricas mínimas

- vagas descobertas por fonte
 - taxa de deduplicação
 - taxa de vagas aproveitáveis
 - taxa de aplicações enviadas
 - taxa de aprovação humana
 - taxa de execução automática bem-sucedida
 - tempo médio por decisão
 - tempo médio por candidatura
 - custo por ciclo agentic
 - conversão por variante de currículo
-

15. Evals e Confiabilidade

15.1 Princípio

Sem avaliação, agente vira show de fumaça.

15.2 Eixos de avaliação

O projeto deve avaliar:

- precisão de extração
- precisão de normalização
- qualidade do match
- qualidade do rationale
- segurança operacional
- robustez do preenchimento
- utilidade das sugestões
- consistência com o histórico

15.3 Regra

Toda skill importante e todo agente relevante devem possuir:

- casos bons;
- casos ruins;
- casos ambíguos;

- critérios explícitos de sucesso e falha.
-

16. Automação de Navegador

16.1 Papel

A automação de navegador será usada como capacidade de execução, não como base identitária do projeto.

16.2 Casos de uso

- abrir vaga
- coletar conteúdo
- interagir com formulário
- capturar evidência
- validar sucesso de envio

16.3 Regras

- sempre preferir caminhos estáveis;
 - automação só com logs e evidência;
 - formularização crítica com aprovação humana;
 - evitar depender de DOM frágil sem fallback.
-

17. Organização do Repositório

17.1 Estrutura macro sugerida

```
olympus-climb/  
  apps/  
    mobile/  
    web-admin/  
  services/  
    api/  
    agent-runtime/  
    worker/  
  packages/  
    core-domain/  
    db/  
    ui/  
    design-tokens/  
    agent-kit/  
    skills/  
    evals/
```

shared/
infrastructure/
docker/
pm2/
scripts/
docs/
architecture/
decisions/
skills/
runbooks/
product/

17.2 Lógica

- **apps/** para interfaces
 - **services/** para processos de execução
 - **packages/** para módulos compartilháveis
 - **infrastructure/** para operação
 - **docs/** como fonte de verdade documental
-

18. Organização de Skills

18.1 Estrutura sugerida

packages/skills/
search-jobs/
open-job-post/
extract-job-requirements/
analyze-company-context/
score-job-fit/
generate-application-assets/
fill-application-form/
save-application-evidence/

18.2 Regra de design

Uma skill boa:

- faz uma coisa;
 - faz bem;
 - explica o que fez;
 - falha de forma compreensível;
 - pode ser testada separadamente.
-

19. Stack de Operação

19.1 O que manter do plano antigo

Eu manteria integralmente:

- PostgreSQL
- Prisma
- Redis
- BullMQ
- PM2
- Artillery
- Expo
- NativeWind
- TypeScript

Porque essas decisões continuam fortes, estáveis e coerentes com teu contexto anterior.

19.2 O que muda de verdade

O que muda não é a base infra.

É o desenho do produto:

- de app-centric para agent-centric;
 - de scraper-centric para decision-centric;
 - de coleta para operação;
 - de automação solta para sistema auditável.
-

20. Diretrizes de Engenharia

20.1 Regra de modularidade

Nada de módulo gigante com responsabilidade difusa.

20.2 Regra de clareza

Toda entidade, agente e skill deve ter propósito declarado.

20.3 Regra de rastreabilidade

Toda decisão importante precisa deixar rastro.

20.4 Regra de evolução

O projeto deve conseguir crescer:

- sem reescrever tudo;
- sem acoplamento circular;
- sem mágica implícita;
- sem depender da memória de uma única pessoa.

20.5 Regra de estabilidade

O sistema deve preferir:

- previsibilidade;
 - contratos claros;
 - schemas fortes;
 - validação explícita;
 - automação observável;
 - intervenção humana quando o risco subir.
-

21. Git e Fluxo de Trabalho

O documento GitFlow anterior continua útil como base disciplinar: `main` protegida, PR obrigatório, zero push direto, versionamento SemVer e branches rastreáveis.

Mas eu faria uma adaptação conceitual:

21.1 Eixos de branch

Em vez de branch por “fase”, eu passaria a aceitar branch por:

- capability
- agent
- skill
- integration
- reliability
- ui
- eval

Exemplos:

- `feature/agent-scout-core`
- `feature/skill-score-job-fit`
- `feature/mcp-gmail-integration`
- `feature/mobile-approval-inbox`
- `feature/evals-match-agent`

21.2 Princípio

A branch representa uma **capacidade evolutiva**, não um degrau cronológico.

22. Documento de Decisões Arquiteturais

22.1 Regra nova

A partir de agora, o Olympus Climb deve manter um conjunto de ADRs.

Cada ADR deve registrar:

- contexto
- decisão
- alternativas consideradas
- trade-offs
- impacto esperado

22.2 ADRs que já existem implicitamente

Os primeiros ADRs deste projeto, na prática, seriam:

1. o projeto será agent-centric;
2. o core será Postgres + Prisma;
3. o app mobile será cockpit, não centro do sistema;
4. skills serão tratadas como unidades formais;
5. MCP será usado como padrão de integração quando fizer sentido;
6. approvals humanos serão obrigatórios em ações sensíveis;
7. o sistema será desenhado para aprender com histórico.

23. Resultado Final Esperado

Quando o Olympus Climb estiver maduro, ele deverá ser capaz de:

- descobrir vagas relevantes com pouco atrito;
 - organizar essas vagas em estrutura canônica;
 - enriquecer contexto de forma útil;
 - ranquear oportunidades com explicação;
 - sugerir estratégia coerente;
 - adaptar ativos de candidatura;
 - executar com segurança;
 - pedir aprovação quando necessário;
 - registrar tudo com rastreabilidade;
 - melhorar com o próprio histórico;
 - servir como laboratório real de IA avançada.
-

24. Sistema de Execução de Engenharia

24.1 Princípio

O Olympus Climb não deve ser desenvolvido por prompting solto, execução *ad hoc* ou tentativas desconectadas.

Sua construção deve ser guiada por especificação, tarefas rastreáveis, contexto persistente e ciclos curtos de validação.

24.2 Spec-Driven Development

Toda mudança relevante deve nascer de uma *spec* clara.

Cada *spec* deve declarar:

- contexto
- objetivo
- escopo
- fora de escopo
- superfícies afetadas
- riscos
- critério de aceite

24.3 Task Manifests

Toda *spec* executável deve ser quebrada em tarefas operacionais menores.

Essas tarefas podem existir em *task manifests*, `tasks.json` ou estrutura equivalente, desde que preservem clareza e rastreabilidade.

Cada tarefa deve registrar:

- objetivo único
- *inputs* necessários
- arquivos, módulos ou superfícies-alvo
- dependências
- restrições
- *Definition of Done*
- evidência de conclusão

24.4 Persistência de Contexto de Projeto

O projeto deve manter uma camada de contexto persistente para reduzir releitura burra, perda de continuidade e decisões contraditórias.

Esse contexto deve incluir, sempre que possível:

- estado atual do sistema
- decisões arquiteturais ativas
- padrões já adotados
- capacidades existentes
- agentes e *skills* disponíveis
- pontos sensíveis do projeto

- próximos passos recomendados

24.5 Regra de Entrega Mínima

A execução deve começar pela menor entrega útil, validável e rastreável.

O fluxo preferencial deve ser:

- definir a *spec*
- quebrar em tarefas menores
- executar o mínimo funcional
- validar o resultado real
- refinar com base em evidência

24.6 Regra de Alvo Explícito

Sempre que possível, a execução deve apontar com precisão:

- o que alterar
- onde alterar
- o que não tocar
- como validar
- qual impacto é esperado

Isso existe para reduzir busca desnecessária, retrabalho e alucinação operacional.24.7 Modelos, Interfaces e Orquestração

Ferramentas como Codex, OpenCode, Claude, MCPs e interfaces equivalentes devem ser tratadas como meios de execução, não como centro da arquitetura.

A escolha de modelo, interface ou fluxo deve obedecer a critérios como:

- aderência à tarefa
- confiabilidade
- custo
- capacidade de manter contexto
- facilidade de rastreamento
- facilidade de substituição futura

24.8 Princípio Operacional Final

O sistema de engenharia do Olympus Climb deve favorecer:

- contexto mínimo suficiente
- tarefas pequenas e claras
- validação frequente
- documentação viva
- continuidade entre execuções
- evolução sem *overengineering*

25. Declaração Final de Arquitetura

O Olympus Climb 2026 será um sistema agentic de operação de carreira, com core de dados confiável, runtime de agentes especialistas, skills plugáveis, integrações padronizadas via tools/MCP quando apropriado, memória operacional, aprovação humana em ações críticas e interfaces voltadas a controle, revisão e execução.

Ele não será desenvolvido como uma sequência de fases de MVP.

Ele será desenhado desde o início como **produto final modular**, ainda que implementado progressivamente.

Esse já é um bom documento-base.

Na próxima etapa, eu recomendo que eu transforme isso em um pacote de três documentos irmãos:

1. **VISION.md** — visão executiva e objetivo do produto
2. **ARCHITECTURE.md** — arquitetura detalhada
3. **CAPABILITY_MAP.md** — mapa de capacidades, agentes e skills

Aí o projeto já nasce com cara de sistema grande de verdade.